



CREMS Team Setup Guide

Run the complete frontend, backend and SQL Server environment

macOS + Windows | React + ASP.NET Core + SQL Server

Outcome

Each team member will run an independent local CREMS environment with a private administrator account and database.

Repository

<https://github.com/kavishc23/CREMS>

CS400 Industry Experience Project



1. How the local system works

Every team member runs the same three parts on their own computer. This avoids sharing database passwords or depending on one person's laptop during development.

Part	Technology	Local address
Frontend	React + TypeScript + Vite	http://localhost:5173
Backend API	ASP.NET Core 10	http://localhost:5080
Database	SQL Server 2022 in Docker	localhost:1433

Important: Do not copy another member's .env file, administrator password, user-secrets store, node_modules folder, .dotnet folder or Docker data volume.

2. Accounts and access required

- A GitHub account that has accepted the private CREMS repository invitation.
- Permission to clone <https://github.com/kavishc23/CREMS>.
- Administrator rights on the computer for installing developer tools.
- A personal local CREMS administrator password that is not shared in chat or committed to Git.

3. Software prerequisites

Software	Required version	macOS	Windows
Git	Current stable	Xcode tools or Git installer	Git for Windows
.NET SDK	10.x	Arm64 for Apple Silicon	x64 for most PCs
Node.js	22 LTS or newer	Arm64 or x64	x64 for most PCs
Docker Desktop	Current stable	Apple/Intel build as appropriate	WSL 2 backend; Linux containers
VS Code	Current stable	C# Dev Kit + ESLint	C# Dev Kit + ESLint

Installation sources: Download developer tools only from their official websites: git-scm.com, dotnet.microsoft.com, nodejs.org, docker.com and code.visualstudio.com.

4. macOS setup

4.1 Verify prerequisites

Run in Terminal or the VS Code terminal:



```
git --version
dotnet --version
node --version
npm --version
docker --version
docker compose version
```

Expected: .NET reports a 10.x SDK, Node.js reports version 22 or newer, and Docker reports both Engine and Compose versions.

4.2 Clone the repository

```
cd ~/Documents
git clone https://github.com/kavishc23/CREMS.git
cd CREMS
```

Authentication: If Git asks for a password, use GitHub CLI (gh auth login), GitHub Desktop, SSH, or a personal access token. GitHub account passwords do not work for Git pushes.

4.3 Restore project dependencies

```
dotnet tool restore
dotnet restore CREMS.slnx
cd src/crems-web
npm install
cd ../../
```

4.4 Start Docker and SQL Server

1. Open Docker Desktop and wait until the Docker engine reports that it is running.
2. From the CREMS repository root, start SQL Server:

```
docker compose up -d
```

3. Confirm the container is healthy:

```
docker compose ps
```

Expected status: cs400-sqlserver-1 is Up and healthy. The first run downloads a large SQL Server image and may take several minutes.

4.5 Configure the local administrator

```
dotnet user-secrets set "BootstrapAdmin:Email" "YOUR_EMAIL" --project
src/CREMS.Api
dotnet user-secrets set "BootstrapAdmin:Password" "YOUR_PRIVATE_PASSWORD" --
project src/CREMS.Api
dotnet user-secrets set "BootstrapAdmin:FullName" "YOUR_NAME" --project
src/CREMS.Api
```

Password rules: Use at least 10 characters with uppercase, lowercase and a number. A symbol is strongly recommended. Do not type the words YOUR_PRIVATE_PASSWORD literally.



5. Windows setup

5.1 Docker Desktop preparation

- Install Docker Desktop with the WSL 2 backend enabled.
- Restart Windows if the installer requests it.
- Open Docker Desktop and verify that it is using Linux containers.
- Wait until Docker Desktop reports that the engine is running.

5.2 Verify prerequisites in PowerShell

Open PowerShell or the VS Code PowerShell terminal:

```
git --version
dotnet --version
node --version
npm --version
docker --version
docker compose version
```

PATH troubleshooting: If a newly installed command is not found, close and reopen VS Code. If it still fails, restart Windows.

5.3 Clone the repository

```
cd $HOME\Documents
git clone https://github.com/kavishc23/CREMS.git
cd CREMS
```

5.4 Restore project dependencies

```
dotnet tool restore
dotnet restore CREMS.slnx
cd src\crems-web
npm install
cd ..\..
```

5.5 Start SQL Server

```
docker compose up -d
docker compose ps
```

Expected status: cs400-sqlserver-1 is Up and healthy. Docker may run the SQL Server image through emulation depending on the computer architecture.



5.6 Configure the local administrator

```
dotnet user-secrets set "BootstrapAdmin:Email" "YOUR_EMAIL" --project  
src/CREMS.Api  
dotnet user-secrets set "BootstrapAdmin:Password" "YOUR_PRIVATE_PASSWORD" --  
project src/CREMS.Api  
dotnet user-secrets set "BootstrapAdmin:FullName" "YOUR_NAME" --project  
src/CREMS.Api
```

6. Run the complete project

Use two terminals and leave both processes running while developing.

Terminal 1 — backend API

```
cd PATH_TO_CREMS  
dotnet run --project src/CREMS.Api
```

Successful startup includes: Now listening on: <http://localhost:5080>

Terminal 2 — frontend

```
cd PATH_TO_CREMS/src/crems-web      # macOS  
cd PATH_TO_CREMS\src\crems-web      # Windows PowerShell  
npm run dev
```

Open <http://localhost:5173> and sign in using the email and password stored in your local user-secrets store.

Keep services running: Do not close either terminal while using CREMS. Press Ctrl+C in each terminal when you want to stop the backend or frontend.

7. First-run verification checklist

- Docker Desktop is running.
- `docker compose ps` shows SQL Server as healthy.
- The backend terminal shows <http://localhost:5080>.
- Opening <http://localhost:5080/api/health> returns a healthy response.
- The frontend terminal shows <http://localhost:5173>.
- The Carpenters-branded login page loads.
- The local administrator can sign in.
- The dashboard displays after authentication.
- The sidebar expands, collapses and opens correctly on smaller screens.



8. Daily start and stop routine

Start work

```
git switch main
git pull
docker compose up -d
dotnet run --project src/CREMS.Api
# In a second terminal:
cd src/crems-web
npm run dev
```

Stop work

Press Ctrl+C in the backend and frontend terminals. SQL Server can remain running. To stop it and preserve the database volume:

```
docker compose stop
```

Do not use docker compose down -v: The -v option deletes the local SQL Server data volume, including the local CREMS database.

9. Working safely with Git

Never develop directly on main. Create a feature branch for each task and open a pull request for team review.

```
git switch main
git pull
git switch -c feature/short-feature-name
# Make and test changes
git add .
git commit -m "Describe the completed change"
git push -u origin feature/short-feature-name
```

- Do not commit real passwords, .env files, user secrets or customer data.
- Pull main before creating a branch.
- Use clear commit messages and small, reviewable changes.
- Ask another member to review the pull request before merging.

10. Troubleshooting

Problem	Resolution
dotnet is not recognized / command not found	Install the .NET 10 SDK, then restart VS Code. Confirm with dotnet --version.
C# Dev Kit reports no SDK	Confirm .NET 10 is installed globally and reload the VS Code window.



Problem	Resolution
Docker command cannot connect	Open Docker Desktop and wait until the engine is running.
SQL Server remains unhealthy	Run docker compose logs sqlserver. Confirm the Compose file is current and port 1433 is available.
Port 1433 is already in use	Stop the other SQL Server/container using that port, then run docker compose up -d again.
Administrator password rejected	Use 10+ characters with uppercase, lowercase and at least one number.
Login page appears but login fails	Confirm the backend is running, the administrator email is correct, and user secrets were set for src/CREMS.Api.
Frontend stays on a loading indicator	Confirm the backend is running. Wait up to eight seconds, then refresh the page.
npm install fails	Confirm internet access and Node.js 22+, then remove no project files; rerun npm install.
GitHub rejects password authentication	Use gh auth login, GitHub Desktop, SSH or a personal access token.

11. Reset the local development environment

Use this only when the local database can be discarded. It removes the SQL Server container and the CREMS Docker volume, then creates a clean database on the next start.

```
docker compose down -v
docker compose up -d
dotnet run --project src/CREMS.Api
```

Data loss: This reset permanently deletes that team member's local CREMS database. It does not affect GitHub or another member's computer.



12. Quick command reference

Task	Command
Update local code	<code>git pull</code>
Restore .NET tools	<code>dotnet tool restore</code>
Restore backend packages	<code>dotnet restore CREMS.slnx</code>
Install frontend packages	<code>npm install (inside src/crems-web)</code>
Start database	<code>docker compose up -d</code>
Check database	<code>docker compose ps</code>
Start backend	<code>dotnet run --project src/CREMS.Api</code>
Start frontend	<code>npm run dev (inside src/crems-web)</code>
Build backend	<code>dotnet build CREMS.slnx</code>
Build frontend	<code>npm run build (inside src/crems-web)</code>
Stop database	<code>docker compose stop</code>